# Bike Store Sales Analytics with SQL

**Prepared by:** Hassan Abdule

**Project summary**

This project analyzes a bike store sales database using SQL to answer practical business questions about customer value, retention, geography, revenue trends, discount impact, and basket behavior. The dataset includes 1,445 customers, 1,615 orders, and 4,722 order line items spanning January 2016 through 2017, with partial 2018 data.

**Core tables: customers → orders → order_items**

| | | |
|---|---|---|
| **1,445**<br>customers | **1,615**<br>orders | **4,722**<br>line items |
| **9.1%**<br>repeat customer rate | **4.38**<br>avg items per order | **$555,559**<br>top product revenue |

## 1. Project overview

**Dataset and schema.** The analysis is built on three related tables linked by foreign keys: **customers**, **orders**, and **order_items**. Revenue is calculated as **quantity × list_price × (1 – discount)**, which was used consistently across spend, product revenue, and monthly trend calculations.

| | |
|---|---|
| **Business domain** | Retail order management / bike store sales analytics |
| **Date coverage** | Jan 2016 through 2017, with partial 2018 records |
| **Entities analyzed** | Customers, orders, order line items, products, cities/states |
| **Primary business themes** | Customer spend, repeat buying, geography, revenue trend, product performance, discount leakage, basket size |
| **Revenue formula** | quantity × list_price × (1 – discount) |
| **SQL techniques applied** | INNER JOIN, LEFT JOIN, GROUP BY, aggregates, HAVING, CTEs, ORDER BY, DATE_FORMAT |

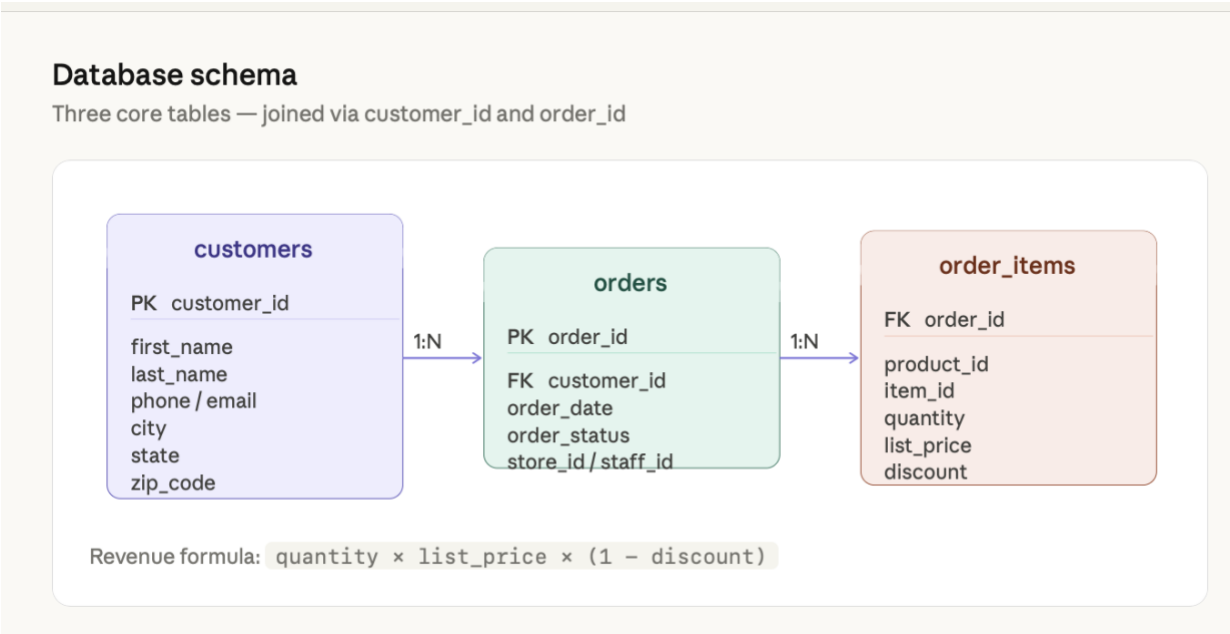


*Figure 1. Database schema used in the analysis.*

## 2. Analytical scope and questions answered

| | |
|---|---|
| **Customer insights** | Top spenders; repeat customers; city/state concentration; customers with no orders |
| **Revenue and sales** | Monthly revenue trend; highest-revenue product; discount loss and margin pressure |
| **Basket analysis** | Average items per order and implications for bundling/upsell behavior |
| **Project outcome** | Convert relational sales data into clear, business-facing insights using SQL |

## 3. Key results snapshot

**The table below summarizes the main findings produced by the SQL queries.** These results are the foundation for the final business insights and recommendations.

| Analysis area | Finding | Why it matters |
|---|---|---|
| **Top customers by spend** | Sharyn Hopkins leads the visible output at $34,808; the top customer cohort also includes Pamelia Newman, Abby Gamble, Lyndsey Bean, and Emmitt Sanchez. | Highlights high-value customers for retention and VIP targeting. |
| **Repeat customers** | 131 customers placed more than one order, which | Shows retention is |

| | | |
|---|---|---|
| | represents a 9.1% repeat customer rate out of 1,445 total customers. | relatively low and worth improving. |
| **Customer geography** | Eight of the top 10 cities by customer count are in New York; Mount Vernon, NY leads with 20 customers and 20 orders. | Supports regional targeting for marketing and logistics. |
| **Customers with no orders** | 0 customers were found without an order, indicating full engagement across the registered customer base. | Confirms the database reflects an active customer base. |
| **Monthly revenue trend** | Revenue rose from about $215K in Jan 2016 to a recorded full-period peak of $378,866 in Jun 2017; later 2018 entries appear partial and were not used for the growth interpretation. | Reveals growth momentum and supports period-over-period tracking. |
| **Average basket size** | Customers purchased an average of 4.38 items per order across 1,615 orders. | Suggests customers already purchase multiple items per order. |
| **Highest discount loss** | Product 7 had the largest discount loss, with $60,440 in revenue given up to discounts. | Identifies a margin leak that pricing teams should review. |
| **Top revenue product** | Product 7 also generated the highest net revenue at $555,559, making it both the strongest earner and the biggest margin risk. | Pinpoints the product most important to top-line performance. |

## 4. Business insights and recommendations

| | |
|---|---|
| **1** | **New York dominates customer geography.** Eight of the top 10 customer cities are in New York, led by Mount Vernon with 20 customers. This concentration suggests that regional campaigns, fulfillment planning, and store-level promotion can be focused more heavily on the New York market. |

| | |
|---|---|
| **2** | **Product 7 is both the top seller and the biggest discount risk.** It generated $555,559 in net revenue but also lost $60,440 to discounts—the highest loss of any product. This is a strong signal for pricing review, discount policy testing, and margin optimization. |

| | |
|---|---|
| **3** | **Revenue showed strong growth through 2017.** From about $215K in Jan 2016 to $378,866 in Jun 2017, revenue increased by roughly 76%. Later 2018 entries appear partial, so the strongest interpretation comes from the full 2016–2017 trend. |
| **4** | **Retention is the clearest growth opportunity.** All customers placed at least one order, but only 131 of 1,445 returned for another purchase. A repeat rate of 9.1% suggests room for email re-engagement, loyalty offers, or post-purchase follow-up. |
| **5** | **Basket size supports bundling and cross-sell strategy.** An average of 4.38 items per order indicates that customers are already buying multi-item baskets. That behavior can be strengthened through bundles, accessory suggestions, or checkout recommendations. |

**Portfolio value. This project demonstrates the ability to join relational tables, calculate business metrics, use CTE-based summaries, and translate SQL output into business recommendations**

## Top 10 customers by spend

QUERY 01

Revenue

Top 10 customers by spend

```sql
SELECT c.customer_id,
  c.first_name, c.last_name,
  SUM(oi.quantity * oi.list_price
    * (1 - oi.discount)) AS total_spend
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
INNER JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY c.customer_id,
  c.first_name, c.last_name
ORDER BY total_spend DESC LIMIT 10;
```

| NAME | TOTAL SPEND |
|---|---|
| Sharyn Hopkins | $34,808 |
| Pamelia Newman | $33,634 |
| Abby Gamble | $32,803 |
| Lyndsey Bean | $32,675 |
| Emmitt Sanchez | $31,926 |

*Figure 2. Top 10 customers by spend query output.*

## Repeat customers

QUERY 02

Retention

### Repeat customers (orders > 1)

```
SELECT c.customer_id,
  c.first_name, c.last_name,
  COUNT(o.order_id)
    AS total_orders
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id,
  c.first_name, c.last_name
HAVING COUNT(o.order_id) > 1;
```

| NAME | ORDERS |
|---|---|
| Pamelia Newman | 3 |
| Sharyn Hopkins | 2 |
| Abby Gamble | 2 |

131 total repeat customers found

## Top 10 cities by customer count

QUERY 03

Geography

**Top 10 cities by customer count**

```sql
SELECT c.city, c.state,
  COUNT(DISTINCT c.customer_id)
    AS total_customers,
  COUNT(o.order_id)
    AS total_orders
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.city, c.state
ORDER BY total_customers DESC
LIMIT 10;
```

| CITY, STATE | CUSTOMERS | ORDERS |
|---|---|---|
| Mount Vernon, NY | 20 | 20 |
| Scarsdale, NY | 17 | 20 |
| Ballston Spa, NY | 17 | 17 |
| Canandaigua, NY | 14 | 16 |

*Figure 4. Top cities by customer count query output.*

## Customers with no orders

QUERY 04

Churn risk

Customers with no orders

```
SELECT c.first_name,
  c.last_name, c.customer_id
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

0

customers found with no orders

Every registered customer has placed at least one order — strong engagement across the full base.

*Figure 5. Customers with no orders query output.*

## Monthly revenue trend

QUERY 05

Monthly revenue trend

Revenue

```
SELECT
  DATE_FORMAT(order_date,
    '%Y-%m') AS month,
  SUM(oi.quantity
    * oi.list_price
    * (1 - oi.discount))
    AS total_revenue
FROM orders o
INNER JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY month
ORDER BY month ASC;
```

| MONTH | REVENUE |
|---|---|
| 2017-06 | $378,866 ↑ peak |
| 2018-04 | $817,922 (partial) |
| 2016-01 | $215,146 |

*Figure 6. Monthly revenue trend query output.*

## Average items per order

QUERY 06

Avg items per order (CTE)

Basket

```sql
WITH order_totals AS (
  SELECT
    SUM(quantity) AS total_items
  FROM order_items
  GROUP BY order_id
)
SELECT
  AVG(total_items) AS avg_items
FROM order_totals;
```

**4.38**

average items per order across 1,615 orders

*Figure 7. Average items per order query output.*

## Product with highest discount loss

QUERY 07

Margin

Product with highest discount loss

```sql
WITH discount_loss AS (
  SELECT product_id,
    SUM(quantity * list_price
      * discount) AS dis_loss
  FROM order_items
  GROUP BY product_id
)
SELECT product_id,
  MAX(dis_loss) AS highest_disc
FROM discount_loss
GROUP BY product_id
ORDER BY dis_loss DESC LIMIT 1;
```

Product ID 7

**$60,440 revenue lost to discounts**

*Figure 8. Product with highest discount loss query output.*

## Top revenue-generating product

QUERY 08

Product

Top revenue-generating product

```sql
WITH product_revenue AS (
  SELECT product_id,
    SUM(quantity * list_price
      * (1-discount))
      AS total_revenue
  FROM order_items
  GROUP BY product_id
)
SELECT *
FROM product_revenue
ORDER BY total_revenue DESC
LIMIT 1;
```

Product ID 7

**$555,559 net revenue**

Also the highest-discounted product — a dual signal worth investigating

*Figure 9. Top revenue-generating product query output.*